

VL-326 Project Report: MAC Instruction and Saturating Arithmetic in RV32I

P19 Implementation on Xilinx Zybo Z7-10

Aayan Garg (B24474)

Microelectronics and VLSI, SCEE

Indian Institute of Technology (IIT) Mandi

Mandi, Himachal Pradesh, India

b24474@students.iitmandi.ac.in

Abstract—This report covers the architectural design, RTL implementation, and experimental evaluation of a custom Digital Signal Processing (DSP) extension built on top of a 5-stage RV32I baseline processor. Three new instructions are introduced: `mac`, a fused multiply-accumulate executed in hardware, and `sadd/ssub` for signed saturating addition and subtraction. The baseline architecture is drawn from the open-source V-FRONT RV32I core [1] and was augmented with a two-cycle sequential multiplier, a third asynchronous read port on the register file, and a deterministic dual-stall hazard detection unit to keep the pipeline correct under multi-cycle execution. The full design was synthesized and physically confirmed on a Xilinx Zybo Z7-10 FPGA using an on-chip Integrated Logic Analyzer (ILA). The central question this project set out to answer was: *how much does a dedicated hardware MAC instruction reduce instruction count and CPI compared to a pure RV32I FIR filter kernel?* For a 4-tap FIR filter benchmark, the results show a 75.5% drop in dynamic instruction count and a 74.0% drop in total execution cycles. These gains come at the cost of a 26.8% rise in LUT utilization and a modest 7.5% drop in maximum operating frequency ($F_{\max}$), confirming that dedicated DSP hardware is a worthwhile architectural trade-off for signal-processing workloads on constrained RISC-V cores.

Index Terms—RISC-V, RV32I, custom ISA extension, multiply-accumulate, saturating arithmetic, DSP acceleration, FIR filter, FPGA implementation, pipeline hazard, Xilinx Zybo Z7-10, RTL design, embedded processor

I. INTRODUCTION AND MOTIVATION

RISC-V has grown into a widely adopted open ISA standard in embedded and edge-computing contexts, largely because of its modular structure and permissive licensing terms [2]. The base integer subset, RV32I, offers a lean and implementation-friendly foundation, but hardware support for multiplication and the saturating arithmetic primitives that DSP workloads routinely depend on is absent by design.

FIR filters are a good example of this gap. Each filter tap reduces to a single multiply-accumulate: $y[n] += h[k] \cdot x[n-k]$. Without a hardware multiplier, a bare RV32I processor must emulate the multiplication through software loops of repeated additions, and preventing overflow demands additional conditional branches on top of that. In resource-constrained settings, this overhead quickly becomes the dominant cost—both in instruction count and in latency.

The RISC-V specification reserves encoding space for non-standard extensions [2], and a number of prior studies have shown that targeted ISA customization can substantially accelerate DSP workloads on small cores [9], [10]. Following that line of work, this project integrates a compact DSP extension into an otherwise unmodified 5-stage RV32I pipeline. The three new instructions are `mac` for hardware multiply-accumulate, and `sadd/ssub` for signed saturation. A 2-cycle sequential multiplier and direct saturation logic in the ALU handle these operations natively. A “dual-stall” hazard scheme manages the resulting multi-cycle latency without breaking in-order pipeline correctness.

The core question is quantitative: *what reduction in instruction count and CPI does the MAC instruction deliver over a pure software RV32I baseline for an FIR kernel?* Both cycle-accurate simulation and on-chip ILA measurements on the Zybo Z7-10 provide the evidence.

The rest of the report is organized as follows. Section II places the work in the context of related ISA extension research. Section III walks through each architectural change and its implementation. Section IV describes the experimental setup and presents the results. Section V closes with a summary and directions for future work.

II. BACKGROUND AND RELATED WORK

A. DSP Limitations of the RV32I Base ISA

RV32I deliberately leaves out multiplication and division, pushing those to the optional RV32M extension [2]. Even with RV32M, getting a full multiply result requires two instructions (`mul` and `mulh`) to cover the 64-bit product, and there is no fused MAC or saturating arithmetic anywhere in the standard. Commercial DSP processors like the TMS320 family [7] and the ARM Cortex-M4 with its SIMD unit [8] have long addressed this with dedicated MAC datapaths and Q-format saturation, but those are proprietary designs with no room for customization.

B. RISC-V ISA Extensibility

The RISC-V specification sets aside four opcode ranges (`custom-0` through `custom-3`) for non-standard exten-

sions, along with encoding conventions for R-, I-, S-, B-, U-, and J-type instructions [2]. Researchers have used this hook for cryptographic acceleration [11], neural-network inference [12], and fixed-point DSP [9]. This project follows the approach of [9]: new operations are mapped into the standard R-type encoding using reserved `funct7` codes, so no toolchain modification is needed and binary compatibility with existing RV32I software is maintained.

C. Hazard Management in Multi-Cycle Pipelines

Adding a variable-latency execution unit to an in-order pipeline creates structural hazards that have to be resolved without introducing errors. The usual options are full pipeline interlocking, forwarding networks, and scoreboarding [6]. When the latency is fixed and predictable, deterministic stalling is the simplest and most area-efficient choice—it needs only a counter or a small state machine rather than a full scoreboard [6]. That is the strategy adopted here: one bubble cycle is injected per `mac` instruction, deterministically.

III. DESIGN AND IMPLEMENTATION

The project ground rules required keeping a strict 5-stage RV32I pipeline (Instruction Fetch [IF], Decode [ID], Execute [EX], Memory Access [ME], and Write-Back [WB]) with no changes to the global clock domain or stage boundaries. Everything had to fit inside that constraint. Work proceeded in two phases: saturating arithmetic first, then the more involved MAC extension.

A. Baseline Architecture and Simulation Environment

The V-FRONT core [1] was chosen as the starting point because it offers a complete, synthesizable 5-stage RV32I implementation with a clean hierarchical RTL structure. Initial work involved sorting out modular dependency problems in the source tree—specifically, fixing `include` paths that did not play well with the Vivado synthesis and simulation flow. Having a verified, cycle-accurate baseline before touching anything else was essential: it served as both a functional reference and a performance baseline for later comparison.

B. Saturating Arithmetic Integration

Saturating arithmetic stops values from wrapping around catastrophically when they exceed the representable range, which is important in fixed-point DSP where intermediate filter products can temporarily exceed full scale before later taps bring them back down.

Two operations were added to the ALU: saturating addition (`sadd`) and saturating subtraction (`ssub`). A dedicated `sat_en` control signal, set by the decoder for both instructions, governs whether the ALU output applies clamping. The adder produces the raw 32-bit sum combinatorially; alongside this, an overflow detector checks the sign bits of both inputs and the result. Overflow is flagged when the two source operands share a sign but the output sign disagrees—the standard two’s-complement overflow condition [6]. When `sat_en` is high and overflow is flagged, a multiplexer replaces the raw sum with the appropriate constant:

`32’h7FFFFFFF` for positive overflow or `32’h80000000` for negative. Fig. 1 shows the simulation waveform confirming that boundary clamping works correctly in both directions.

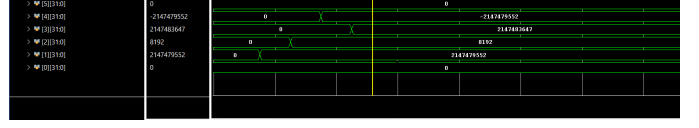


Fig. 1. Simulation waveform for saturating arithmetic boundary clamping. Both positive and negative overflow events result in the output holding at the appropriate 32-bit saturation constant instead of wrapping.

C. Multi-Cycle MAC and Register File Expansion

The standard RV32I R-type format supports two source registers (`rs1`, `rs2`) and one destination (`rd`). The MAC operation, $rd \leftarrow rd + (rs1 \times rs2)$, needs to read three operands per cycle: the two multiplicands and the current accumulator. That is one read port more than the baseline register file provides.

The fix was to add a third asynchronous read port to the register file. The decoder was updated to route `instr[11:7]` (the `rd` field) to this new port’s address input (`rs3_addr`), and the corresponding data output (`rs3_data`) is passed to the EX stage as the accumulator operand.

One correctness issue arises when back-to-back instructions target the same accumulator register: the updated value being written in WB may not yet be in the register file array when the next `mac` reads via `rs3`, which is a classic Read-After-Write (RAW) hazard. This was addressed with write-through bypassing inside the register file—if write-enable is asserted and the write address matches `rs3_addr`, the incoming write data is forwarded directly to the `rs3` read port, bypassing the storage array. The `mac` instruction therefore always sees the most recent accumulator value regardless of what else is in the pipeline.

Functional correctness of the MAC operation alone is shown in Fig. 2. End-to-end verification with concurrent saturation is in Fig. 3.

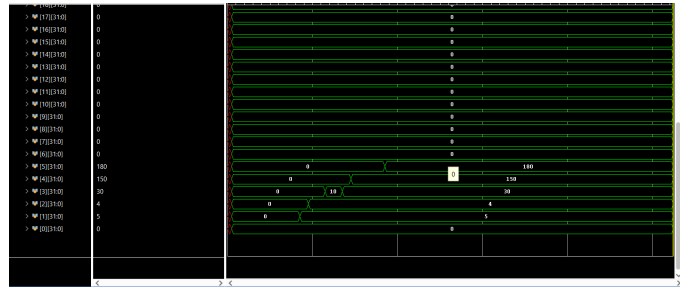


Fig. 2. Simulation trace confirming correct `mac` behavior. The accumulator (`rd`) is updated to $rd + (rs1 \times rs2)$ each time the instruction is issued.

D. Dual-Stall Hazard Management

The sequential multiplier takes two clock cycles: the product $rs1 \times rs2$ is registered at the end of the first cycle and feeds

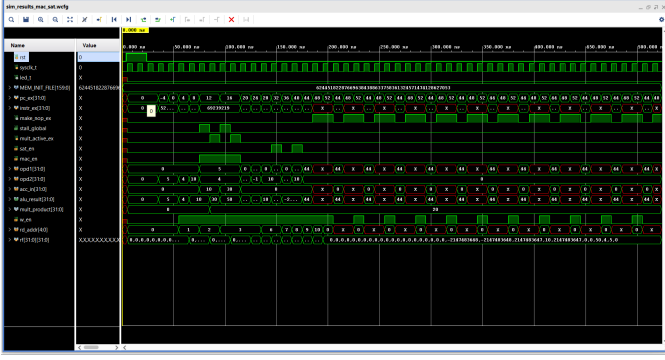


Fig. 3. End-to-end simulation waveform with MAC execution and saturation clamping active simultaneously, confirming the two extensions do not interfere.

into the accumulating adder at the start of the second. Inside the nominally single-cycle EX stage, this creates a structural hazard—the multiplication result is not ready in time for the ME stage to proceed on the very next cycle.

Rather than slowing the global clock to accommodate the multiplier (which would hurt every non-MAC instruction), a centralized Dual-Stall hazard detection unit was built. It watches a `mult_active_ex` signal. When a `mac` enters the EX stage, the multiplier starts and the unit simultaneously raises `stall_global` for exactly one cycle. This signal freezes the program counter and all four inter-stage pipeline registers (IF/ID, ID/EX, EX/ME, ME/WB). One cycle later the stall drops, the registered product is available, and the ALU finishes the accumulation combinatorially. The net result is that each `mac` occupies the EX stage for two cycles, injecting one bubble, so its effective CPI is 2.0.

This scheme is fully deterministic: no instruction replay, no speculation, and no out-of-order bookkeeping. The full modified 5-stage datapath—with the third register file port, the saturating MAC unit, and the dual-stall paths—is shown in Fig. 4.

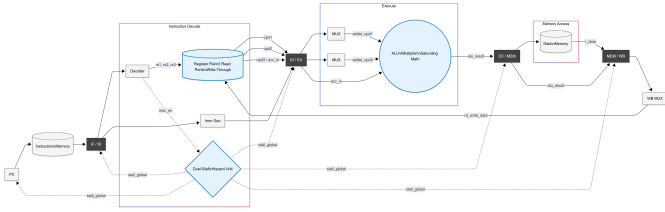


Fig. 4. Modified 5-stage RV32I pipeline datapath. New additions include the three-port register file, the saturating MAC unit with its 2-cycle sequential multiplier, and the dual-stall hazard paths from EX to the PC and all pipeline registers.

E. Instruction Set Encodings

All three custom instructions use the standard 32-bit R-type format, reusing the Integer Register-Register opcode (0110011) and `funct3` field (000). Distinct, previously-reserved `funct7` sequences ensure no collision with standard

RV32I or RV32M encodings. Table I lists the full encoding for each instruction.

TABLE I
R-TYPE INSTRUCTION ENCODINGS FOR THE CUSTOM DSP EXTENSION

Instr.	funct7	rs2	rs1	funct3	rd	opcode
sadd	0000001	src2	src1	000	dest	0110011
ssub	0100001	src2	src1	000	dest	0110011
mac	0000010	src2	src1	000	dest	0110011

Because standard RISC-V GCC does not recognize these mnemonics, the custom machine code is injected directly via the assembler `.word` directive. For instance, `0x042081b3` encodes `mac x3, x1, x2` (`funct7`: 0000010, `rs2`: 00010, `rs1`: 00001, `funct3`: 000, `rd`: 00011, `opcode`: 0110011). This sidesteps the need to rebuild the GCC toolchain while keeping bit-exact encoding, consistent with approaches used in earlier RISC-V custom extension work [9].

IV. METHODOLOGY AND RESULTS

A. Benchmark Workload

A 4-tap FIR filter kernel was chosen as the primary benchmark because it captures the core computational pattern of DSP workloads: a fixed-length loop of multiply-accumulate operations over a known coefficient set. The kernel computes $y[n] = \sum_{k=0}^3 h[k] \cdot x[n-k]$, using a scalar weight $h = 3$ and a scalar sample $x = 5$ for repeatable verification.

Two hand-assembled RV32I versions were written. The *baseline* emulates multiplication with a software inner loop of repeated additions (Fig. 5). The *optimized* version replaces that inner loop with a single `mac` instruction injected via `.word` (Fig. 6). Both produce the same numerical result, so the comparison is direct and fair.

At the binary level, eliminating the inner loop cuts the static code size as well. The baseline memory image (`fir_baseline.mem`) is 40 bytes; the optimized image (`fir_custom.mem`) is 32 bytes—a **20% reduction in static code size**. Smaller code means less instruction-cache pressure, which carries secondary benefits in larger embedded deployments.

B. FPGA Synthesis and On-Chip Verification

The full RTL was synthesized and implemented for the Xilinx Zybo Z7-10 (Artix-7 fabric) in Vivado 2025.2. To get stable timing closure during initial verification and to decouple the design from the board's 125 MHz oscillator jitter, a Mixed-Mode Clock Manager (MMCM) IP was used to bring the core clock down to a stable 25 MHz. Synthesis was also run without timing constraints to find the unconstrained $F_{\max}$ reported in Table II.

Key pipeline signals were tagged with (`*mark_debug = "true"`) and routed to an embedded ILA core so the dual-stall logic could be checked directly in hardware. Hardware button inputs served as trigger sources, allowing cycle-accurate pipeline captures during `mac` execution. The ILA trace in

```

.global _start
.text

_start:
    # --- INITIALIZATION ---
    addi x8, x0, 4      # x8 = 4 (outer loop counter
                        # , N=4 taps)
    addi x3, x0, 0      # x3 = 0 (accumulator: y[n])
    addi x1, x0, 3      # x1 = 3 (coefficient: h)

outer_loop:
    addi x2, x0, 5      # x2 = 5 (sample: x, inner
                        # loop counter)

inner_loop:
    add x3, x3, x1      # y[n] += h (software
                        # multiply via addition)
    addi x2, x2, -1     # decrement inner counter
    bne x2, x0, inner_loop

    addi x8, x8, -1     # decrement outer counter
    bne x8, x0, outer_loop

end:
    jal x0, end        # spin-halt

```

Fig. 5. Baseline RV32I FIR kernel. Multiplication is emulated by a software inner loop of repeated additions, inflating instruction count and cycle count relative to the hardware-accelerated version.

```

.global _start
.text

_start:
    # --- INITIALIZATION ---
    addi x8, x0, 4      # x8 = 4 (loop counter: N=4
                        # taps)
    addi x3, x0, 0      # x3 = 0 (accumulator: y[n])
    addi x1, x0, 3      # x1 = 3 (coefficient: h)
    addi x2, x0, 5      # x2 = 5 (sample: x)

loop:
    # mac x3, x1, x2 --> x3 = x3 + (x1 * x2)
    # Injected as raw machine code (funct7=0000010,
    # opcode=0110011)
    .word 0x042081b3

    addi x8, x8, -1     # decrement loop counter
    bne x8, x0, loop    # repeat for all N taps

end:
    jal x0, end        # spin-halt

```

Fig. 6. Optimized FIR kernel using the custom `mac` instruction via the `.word` directive. The entire inner software multiplication loop is replaced by a single hardware instruction.

Fig. 7 shows the Program Counter holding for exactly one cycle per `mac` instruction—physical proof that the hazard logic works as intended in the deployed bitstream.

C. Simulation Traces

Cycle-accurate behavioral simulations were run for both kernel variants to get exact instruction and cycle counts. The waveform for the baseline (Fig. 8) and for the modified architecture (Fig. 9) form the empirical basis for the performance analysis in Section IV-E.

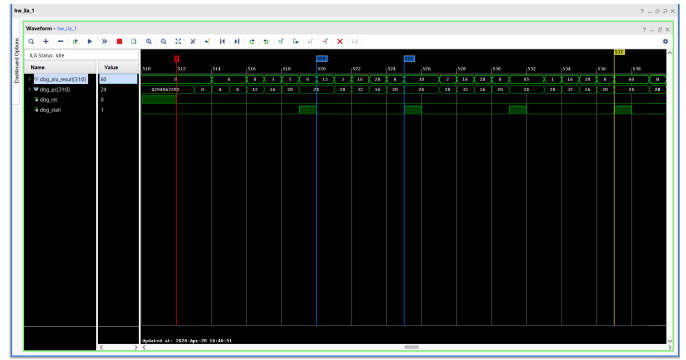


Fig. 7. ILA capture from the Xilinx Zybo Z7-10. The Program Counter is held constant for one clock cycle during `mac` execution, confirming correct operation of the dual-stall hazard detection unit in silicon.

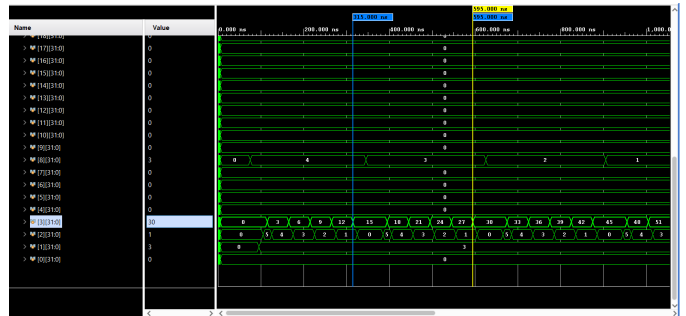


Fig. 8. Simulation waveform of the baseline RV32I pipeline running the software-emulated FIR kernel. The long execution trace reflects the repeated addition loops that stand in for multiplication across all four filter taps.

D. Hardware Resource Utilization

Table II compares post-implementation resource utilization and timing between the baseline and modified cores under identical synthesis constraints. ILA overhead is excluded from both figures to keep the architectural comparison clean.

Adding the three-port register file, the 2-cycle sequential multiplier, and saturation logic raised LUT usage by 26.8% and flip-flop count by 10.4%. Block RAM utilization was unchanged. Vivado inferred and mapped the sequential multiplier onto three DSP48E1 slices, confirming the intended hardware mapping. The $F_{\max}$ of the modified core dropped 7.5%—from 50.25 MHz to 46.48 MHz—due to the extended critical path from the saturation output multiplexer and the added register file read port.

TABLE II
POST-IMPLEMENTATION RESOURCE UTILIZATION ON XILINX ZYBO Z7-10

Resource	Baseline RV32I	Modified (P19)	Change
LUT Count	2109	2674	+26.8%
FF Count	1927	2128	+10.4%
BRAM	12	12	$\pm 0.0\%$
DSP48E1	0	3	N/A
$F_{\max}$ (MHz)	50.25	46.48	−7.5%

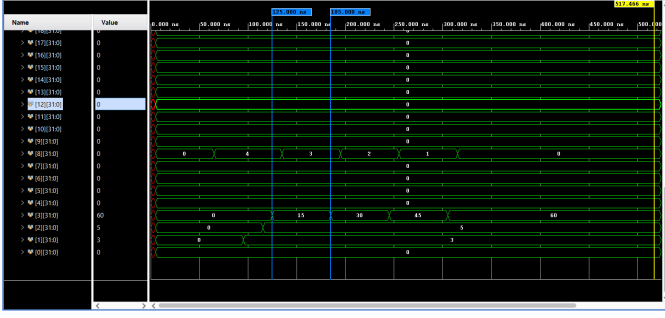


Fig. 9. Simulation waveform of the DSP-extended pipeline running the hardware-accelerated FIR kernel. The much shorter trace reflects the replacement of four inner loops with four `mac` instructions.

E. Performance Analysis

Table III summarizes the key performance numbers from the cycle-accurate simulations of both kernel variants.

TABLE III
PERFORMANCE COMPARISON FOR THE 4-TAP FIR FILTER KERNEL

Metric	Baseline RV32I	Modified (P19)	Change
Instruction Count	98	24	−75.5%
Execution Cycles	108	28	−74.0%
Average CPI	1.10	1.16	+5.4%

The baseline took 108 cycles to finish the 4-tap FIR kernel, issuing 98 instructions across four outer loop iterations. The modified architecture finished the same computation in 28 cycles with only 24 instructions. That directly answers the research question: a **75.5% reduction in dynamic instruction count** and a **74.0% reduction in total execution cycles** over the pure RV32I baseline.

Average CPI increased from 1.10 to 1.16 (5.4%), as expected from the one-cycle structural stall per `mac`. Each `mac` runs at CPI = 2.0 while all surrounding instructions run at CPI = 1.0 as before. The slight average-CPI rise reflects the fact that only 4 of the 24 instructions in the optimized kernel are `mac` instructions.

The overall trade-off is clearly favorable. A 7.5% $F_{\max}$ penalty and a 5.4% average-CPI rise are minor costs against a 74% wall-clock reduction. From an energy perspective, the steep drop in active cycles implies a proportional reduction in dynamic power for this workload—a significant consideration for battery-powered edge devices.

F. Limitations and Threats to Validity

A few factors limit how far these results generalize. The benchmark is a single, scalar 4-tap FIR; larger filters with vector operands, time-varying coefficients, or block-floating-point representations may produce different instruction-count ratios. The on-chip verification runs at a fixed 25 MHz rather than the unconstrained $F_{\max}$, so cycle counts are implementation-independent but any wall-clock comparison must account for the 7.5% frequency gap between the two cores. Additionally,

the baseline lacks the RV32M multiply extension; a more even-handed comparison for future work would pit the custom `mac` against RV32M `mul+add` sequences, which would still carry separate-instruction overhead but would remove the software multiplication loop entirely. Finally, the utilization numbers are specific to the Artix-7 FPGA family; results on other process nodes or in an ASIC flow may differ.

V. CONCLUSION

This project designed, implemented, and physically verified a custom DSP extension on a 5-stage RV32I processor, adding a hardware multiply-accumulate instruction (`mac`) and saturating arithmetic primitives (`sadd`, `ssub`). The modified core was synthesized and tested on a Xilinx Zybo Z7-10, with on-chip ILA measurements confirming that the deterministic dual-stall hazard mechanism works correctly in silicon.

For a 4-tap FIR filter, the `mac` instruction delivers a **75.5% reduction in dynamic instruction count** (from 98 to 24) and a **74.0% reduction in total execution cycles** (from 108 to 28) compared to a pure RV32I software-emulation baseline. The hardware cost is a 26.8% increase in LUT usage, a 10.4% rise in flip-flop count, and a 7.5% drop in $F_{\max}$ —all modest and reasonable given the speedup delivered.

The results reinforce a broader design principle: for workloads built around a small set of well-defined, repetitive operations, adding a few custom multi-cycle instructions with deterministic stalling can yield dramatic throughput improvements without any fundamental restructuring of the host pipeline. This fits naturally within the RISC-V philosophy of composable, purpose-fit hardware.

Several extensions would be worth pursuing. Replacing the 2-cycle sequential multiplier with a fully pipelined Booth multiplier [7] would eliminate the stall cycle and recover the CPI penalty, at the cost of additional logic resources. Extending `mac` to handle 16-bit packed SIMD operands would let two filter taps execute per cycle. Finally, benchmarking against a proper RV32M `mul+add` sequence would isolate the benefit of instruction fusion from the benefit of hardware multiplication alone.

REFERENCES

- [1] K. Dikmen, “V-FRONT: A 5-stage pipelined RV32I core,” GitHub Repository, <https://github.com/kagandikmen/V-FRONT>, Accessed: April 2026.
- [2] A. Waterman and K. Asanović, “The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA,” *RISC-V International*, Document Version 20191213.
- [3] A. Garg, “RV32I-Custom-DSP-Core,” GitHub Repository, <https://github.com/ron-in-boots/RV32I-Custom-DSP-Core>, Accessed: April 2026.
- [4] Digilent Inc., “Zybo Z7 Board Reference Manual,” *Digilent Reference*, 2019.
- [5] B. Paul, “VL-326: Fundamentals of Processor Architecture and SoC Design — Course Project Handout,” Indian Institute of Technology (IIT) Mandi, Academic Year 2025–26.
- [6] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Waltham, MA, USA: Morgan Kaufmann, 2019.
- [7] S. K. Mitra, *Digital Signal Processing: A Computer-Based Approach*, 4th ed. New York, NY, USA: McGraw-Hill, 2011.
- [8] ARM Limited, “ARM Cortex-M4 Processor Technical Reference Manual,” ARM Document 100166_0001_00_en, 2015.

- [9] F. Scholl and J. Noguera, "Extending RISC-V for DSP applications: A case study on fixed-point arithmetic," in *Proc. IEEE Int. Conf. on Application-Specific Systems, Architectures and Processors (ASAP)*, 2021, pp. 1–8.
- [10] A. G. Waterman, Y. Lee, D. A. Patterson, and K. Asanović, "The RISC-V Compressed Instruction Set Manual," *EECS Department, University of California, Berkeley*, Tech. Rep. UCB/EECS-2011-62, 2011.
- [11] B. Marshall, G. R. Newell, D. Page, M. J. B. Robshaw, and C. Wolf, "The design of scalar AES instruction set extensions for RISC-V," *IACR Trans. Cryptographic Hardware and Embedded Systems*, vol. 2021, no. 1, pp. 109–136, 2020.
- [12] G. Ottavi *et al.*, "A mixed-precision RISC-V processor for extreme-edge DNN inference," in *Proc. IEEE Int. Symp. on Circuits and Systems (ISCAS)*, 2020, pp. 1–5.